Python Engineering - Reference Guide

CLASS 16 - Testing & Project Structure

Guía de consulta rápida y ampliada sobre tests básicos, assertions, tolerancias numéricas, organización de archivos y ejecución.

1. test

Un test es código cuyo propósito es comprobar automáticamente que otro código se comporta como esperamos.

```
def test_something():
    result = function_under_test()
    assert result == expected
```

2. Nombre test_...

Usamos nombres como `test_euler_step` porque describen claramente que la función es una prueba y qué comportamiento verifica.

```
def test_euler_step():
    ...
```

3. Arrange - Act - Assert

Fase	Pregunta
Arrange	¿Qué objetos, datos y condiciones preparo?
Act	¿Qué función o método ejecuto?
Assert	¿Qué resultado espero y cómo lo compruebo?

```
# Arrange
state = VerticalState(h=100.0, v=20.0)

# Act
state.euler_step(0.1)

# Assert
assert np.allclose(
    state.state_vector(),
    [102.0, 19.019, 0.1]
)
```

4. assert

`assert condition` comprueba una expresión booleana.

```
assert condition
assert condition, 'message if it fails'
```

Si la condición es True, el programa continúa. Si es False, Python lanza `AssertionError`.

5. AssertionError

```
assert 2 + 2 == 5, 'Math changed!'
# AssertionError: Math changed!
```

En testing, un AssertionError suele significar **FAIL**: el código se ejecutó, pero el resultado no coincidió con lo esperado.

6. np.allclose()

Compara arrays numéricos permitiendo pequeñas diferencias de coma flotante.

```
np.allclose(actual, expected)
np.allclose(actual, expected, rtol=..., atol=...)
```

Argumento	Significado
actual	Valores calculados
expected	Valores de referencia
rtol	Tolerancia relativa
atol	Tolerancia absoluta

Para este bootcamp normalmente dejamos las tolerancias por defecto salvo que el problema físico exija otra cosa.

7. Igualdad exacta vs tolerancia

```
actual == expected
# exact element-by-element comparison

np.allclose(actual, expected)
# numerical closeness
```

Para enteros o strings, igualdad exacta tiene sentido. Para resultados numéricos con floats, una tolerancia suele ser más apropiada.

8. Test de excepción esperada

```
try:
    operation_that_should_fail()
except ExpectedError:
    pass
else:
    assert False, 'Expected exception was not raised'
```

El test pasa si aparece la excepción esperada y falla si el código continúa normalmente.

9. PASS / FAIL / ERROR

Estado	Interpretación
PASS	El test termina y todas las assertions se cumplen
FAIL	El test termina con AssertionError
ERROR	Aparece una excepción inesperada durante el test

10. test.__name__

Las funciones son objetos. `test.__name__` nos da su nombre para imprimir resultados.

```
def test_example():
    pass

print(test_example.__name__)
# test_example
```

11. Lista de funciones TESTS

```
TESTS = [
    test_initial_state,
    test_euler_step,
]
```

Guardamos referencias a funciones, no resultados. Por eso no escribimos `test_initial_state()` dentro de la lista.

12. except Exception en el runner

En código de aplicación evitamos capturas amplias por defecto. En un test runner pequeño puede ser útil capturar errores inesperados para informar de ellos y continuar con el resto de tests.

```
except Exception as error:
    print('ERROR')
```

La clave es que **no lo ocultamos**: mostramos el tipo y el mensaje.

13. Path(__file__).resolve().parent.parent

```
base_dir = Path(__file__).resolve().parent.parent
```

En un archivo dentro de `tests/`:

```
test_file_io.py
    parent        -> tests/
    parent.parent -> ex24_testing_project_structure/
```

14. Path.unlink()

Elimina un archivo.

```
test_file.unlink()
test_file.unlink(missing_ok=True)
```

`missing_ok=True` evita error si el archivo ya no existe. Lo usamos para limpiar archivos temporales de test.

15. __init__.py

En este ejercicio es un archivo vacío dentro de `tests/` que nos permite importar el directorio como paquete de Python.

```
tests/
|-- __init__.py
|-- test_vertical_state.py
`-- test_file_io.py
```

> Más adelante veremos paquetes con más profundidad; aquí solo necesitamos esta función estructural.

16. Código de aplicación vs código de test

Aplicación	Tests
Calcula, simula, guarda, carga	Comprueba que esas operaciones funcionan
Debe ser reutilizable	Debe ser claro y repetible
No conoce los tests	Importa y prueba la aplicación
vertical_state.py / file_io.py	tests/test_*.py

17. main.py

`main.py` demuestra o ejecuta la aplicación. No debería contener las assertions de test del proyecto.

```
main.py -> use the software
tests/  -> verify the software
```

18. run_tests.py

Es nuestro runner educativo. Su trabajo es descubrir manualmente la lista de tests, ejecutarlos y resumir el resultado.

> En proyectos reales, frameworks como pytest automatizan gran parte de esto. Aquí lo construimos a mano para entender el mecanismo.

19. Un test = un comportamiento claro

Cuanto más específico es un test, más fácil es interpretar un fallo.

```
good:
test_initial_state()
test_euler_step()
test_missing_file_raises()

less clear:
test_everything()
```

20. Repetibilidad

Ejecutar el mismo test en las mismas condiciones debería producir el mismo resultado. Evita dependencias innecesarias de archivos manuales, orden de ejecución o estado dejado por otro test.

21. Independencia

Un test no debería necesitar que otro test se ejecute primero. Cada test prepara su propio estado.

```
test A -> arrange its own data
test B -> arrange its own data
```

22. Convenciones de estructura

```
project/
|-- application modules
|-- main.py
|-- data/
|-- tests/
`-- run_tests.py
```

La estructura comunica intención: incluso antes de abrir los archivos, otra persona puede saber dónde está cada responsabilidad.

23. Cheat sheet

```
assert condition, 'message'

np.allclose(actual, expected)

def test_name():
    # Arrange
    ...
    # Act
    ...
    # Assert
    assert ...

PASS  -> expectations satisfied
FAIL  -> AssertionError
ERROR -> unexpected exception
```